AI for Science Applied to Tax Planning: An Agent-Governed, Human-in-the-Loop Architecture for Strategy Discovery and Audit Review

Alejandro Reynoso
Chief Scientist, DEFI Capital

June 10, 2026

Abstract

This paper develops an agent-governed architecture for AI-assisted tax planning inspired by the *AI for Science* (AI4S) paradigm. Rather than treating a large language model as a tax oracle, the system places generative AI inside a closed-loop advisory workflow in which candidate strategies are proposed, validated, simulated, stress-tested, reviewed, and either rejected, revised, or retained under explicit governance criteria. The architecture is implemented in a companion Colab notebook that combines a synthetic tax code, a detailed taxpayer profile, a deterministic tax simulation engine, LLM-supported strategy generation, LLM-supported legal screening, scenario stress testing, a simulated human-advisor review layer, and a supervisory `TaxEvaluationAgent` that decides whether the research process should continue or stop.

The paper specifies the workflow, software architecture, mathematical core, agent decision protocol, legal and numerical safeguards, LLM interfaces, stress scenarios, and audit artifacts required for a reproducible AI-assisted tax-planning discovery system. The central methodological distinction is between *generation* and *determination*: the LLM may generate planning hypotheses and professional commentary, but the tax liability itself is computed by a deterministic `compute_tax()` engine. This separation reduces the risk of hallucinated tax outcomes and preserves a reviewable audit trail.

The contribution is methodological. The system is not a substitute for professional tax advice and does not purport to represent actual tax law. Instead, it provides a blueprint for using AI in tax, accounting, and advisory services under controlled conditions: explicit rulebooks, structured JSON interfaces, deterministic calculations, legal gates, audit-risk filters, professional skepticism, stress tests, and decision logs. The resulting architecture demonstrates how AI4S can be adapted from scientific discovery to professional-services governance. Its value lies not in automating tax judgment away, but in making tax-planning exploration more systematic, transparent, auditable, and accountable.

Keywords: artificial intelligence for science; tax planning; audit governance; agentic AI; human-in-the-loop review; professional services; deterministic simulation.

1. Introduction

Tax planning is an advisory problem conducted under legal, numerical, professional, and client-specific constraints. A planning strategy that appears attractive because it lowers tax liability may nevertheless be unsuitable if it is legally unsupported, inconsistent with the taxpayer's facts, overly complex, fragile under alternative scenarios, exposed to audit risk, or difficult for a professional advisor to defend. For this reason, credible tax-planning technology must distinguish sharply between a tax idea, a simulated tax result, a legally valid recommendation, and an advisor-approved course of action.

This paper contends that the AI4S paradigm is valuable for tax planning precisely because it forces this distinction to be made explicit. In AI4S, artificial intelligence is not merely a tool for answering questions; it is a participant in a research cycle. It helps represent observations, propose hypotheses, test alternatives, interpret failures, and refine the next search direction. In scientific domains, this architecture has been used to accelerate discovery by placing AI inside experimental loops. In tax and accounting, however, the analogy

must be adapted carefully. The relevant object of discovery is not a physical molecule, a material, or a biological structure. The object is a legally and numerically coherent planning strategy that remains suitable under professional review.

The architecture developed here applies the AI4S grammar to tax planning. A synthetic tax code defines the rule environment. A taxpayer profile defines the case facts and constraints. A deterministic simulation engine computes baseline and adjusted tax outcomes. An LLM generates candidate planning strategies using an approved adjustment vocabulary. A legal validity gate screens the proposed strategies. The best simulated strategy is stress-tested under adverse scenarios. A simulated human advisor, modeled as a skeptical CPA, reviews the candidate for suitability, audit risk, and client communication. Finally, a supervisory `TaxEvaluationAgent` decides whether the system should stop, continue, or halt after the maximum number of rounds.

The design is deliberately conservative. It does not ask the LLM to compute tax liability directly. It does not accept a strategy merely because the LLM describes it persuasively. It does not treat tax savings as the only objective. Instead, it separates creativity, computation, legal review, stress testing, professional skepticism, and final governance into different modules. This separation is the central methodological contribution of the paper.

The implementation is contained in a companion Colab notebook. The executable implementation is organized into nine cells: setup and Claude client; synthetic tax code; taxpayer profile; deterministic tax engine; the optimization loop; the human advisor agent; the evaluation agent; orchestrator; and final dashboard. The notebook also contains introductory Markdown that frames the AI4S analogy. The resulting workflow is compact enough for classroom use but rich enough to illustrate the core institutional logic of governed AI in tax and accounting.

The paper is organized as follows. Section 2 presents the end-to-end workflow. Section 3 describes the software architecture and the separation between the inner optimization loop and the outer evaluation agent. Section 4 specifies the mathematical and computational core used for income aggregation, deductions, tax calculation, credits, and strategy comparison. Section 5 formalizes the decision protocol. Section 6 describes the numerical and legal safeguards. Section 7 explains the LLM integration. Section 8 presents the stress-testing scenarios. Section 9 discusses contributions, limitations, and future research. The appendix documents the companion notebook and includes the workflow diagram.

The intended contribution is methodological. The paper does not claim that the synthetic tax code is a substitute for actual federal, state, or international tax law. Nor does it claim that an LLM can replace a licensed tax advisor. Instead, it demonstrates how AI can be placed inside a disciplined professional workflow: one that records assumptions, computes outcomes reproducibly, subjects recommendations to adversarial review, and preserves an audit trail for every continuation or stopping decision.

2. Workflow Overview

The workflow is a closed-loop tax-planning discovery procedure. It begins with a synthetic tax environment and a taxpayer profile, generates candidate strategies, validates them, simulates their tax effects, stress-tests the best candidate, obtains professional-style review, and then decides whether to stop or continue. The loop is designed to prevent a familiar weakness of AI-assisted professional work: the promotion of plausible language into operational advice without adequate computation, validation, or review.

Table 1 summarizes the ten-stage workflow implemented in the notebook.

The workflow contains two nested loops. The inner loop is implemented by `TaxOptimizationLoop`. This loop performs one full planning cycle: strategy generation, legal review, simulation, stress testing, feedback, and seed generation. The outer loop is implemented by `TaxEvaluationAgent`. This agent governs the full research process across up to three planning rounds. It receives the result of each inner loop,

Table 1: Ten-stage AI4S tax-planning workflow.

Stage	Description
1. Synthetic Tax Code	Encodes brackets, deductions, credits, AMT-style rules, NIIT, self-employment tax, contribution limits, and underpayment-penalty parameters.
2. Taxpayer Profile	Defines the client facts: wages, consulting income, rental activity, capital gains, dividends, deductions, withholding, dependents, constraints, and preferences.
3. Baseline Simulation	Computes baseline tax liability using deterministic arithmetic.
4. Strategy Generation	Claude proposes exactly five bounded tax-planning strategies using approved adjustment keys.
5. Legal Validity Gate	Claude assigns each strategy a `LEGAL`, `BORDERLINE`, or `ILLEGAL` verdict under the synthetic rules.
6. Tax Impact Simulation	Deterministic `compute_tax()` engine calculates each non-`ILLEGAL` strategy and ranks results by simulated savings.
7. Stress Testing	Best strategy is tested under the code's three scenarios: rate increase, deduction cap, and income spike.
8. Feedback and Seeds	LLM provides diagnostic feedback and proposes next-cycle search seeds.
9. Human Advisor Review	Claude simulates Margaret Chen, CPA, who reviews the top strategy for suitability, defensibility, and client communication.
10. Evaluation Decision	`TaxEvaluationAgent` decides whether to continue, stop successfully, or halt after maximum rounds.

obtains a simulated CPA review, evaluates savings and risk, records the decision, and determines whether additional search is justified.

The critical control point is the legal-and-governance gate. A strategy is not accepted solely because it reduces the simulated tax liability. It must be expressible in the approved adjustment vocabulary, compatible with the synthetic tax code, acceptable under the legal validity gate, non-fragile under stress testing, and suitable under CPA-style review. This architecture converts tax planning from a one-shot prompt into a governed research process.

The workflow also imposes a hierarchy of evidence. A generated strategy begins as a hypothesis. It gains evidentiary status only after it has been encoded as structured adjustments. It gains additional status after deterministic simulation. It gains further credibility if it survives stress tests. It becomes professionally plausible only after human-advisor review. Finally, it becomes a recommended candidate only if the supervisory evaluation agent records a stop-success decision. This hierarchy is central to the AI4S mapping: the model discovers, but governance decides.

3. Software Architecture

The implementation is organized around four layers: a rule environment, a deterministic tax engine, an inner agentic optimization loop, and an outer governance agent. This separation is not merely a software convenience. It is the primary mechanism by which the notebook preserves auditability. Each layer has a specific responsibility and a bounded authority.

3.1. Synthetic Tax Code

The `TAX_CODE` dictionary defines the rule environment. It includes ordinary income brackets, capital-gains brackets, standard deductions, retirement contribution limits, HSA contribution limits, IRA phaseouts, QBI rules, itemized deduction rules, SALT caps, charitable deduction limits, credits, self-employment tax, NIIT,

AMT-style rules, capital-loss carryforward rules, and safe-harbor underpayment-penalty parameters. Audit risk is not a tax code field in the implementation; it is supplied by generated strategies and reviewed by the governance agents. The purpose of this object is to create a controlled legal universe in which the model can safely experiment.

The synthetic tax code is not presented as actual tax law. Its methodological purpose is to separate legal structure from LLM language. The LLM is not permitted to invent tax rules during strategy evaluation. Candidate strategies must be judged inside the encoded environment. This makes the experiment reproducible and prevents a common failure mode of generative systems: fluent but unsupported claims about legal consequences.

3.2. Taxpayer Profile

The `TAXPAYER` dictionary defines the client case. It includes W-2 income, consulting income, rental income and expenses, dividends, long-term capital gains, short-term capital gains, unrealized losses, retirement contributions, HSA contributions, IRA contributions, charitable giving, mortgage interest, property tax, state income tax, prior-year tax, current withholding, dependents, state, and client preferences. The case is Alex and Jordan Chen, married filing jointly, with two dependents, California residence, moderate general risk tolerance, low audit-risk preference, a $50,000 liquidity limit, and a this-year planning horizon.

This level of detail matters because tax planning is fact-dependent. A strategy that is valuable for one household may be irrelevant or harmful for another. The taxpayer profile supplies the factual substrate over which the strategy search operates. It also records client constraints such as liquidity needs, audit-risk tolerance, preference for simplicity, and conservative advisory posture. These preferences influence whether a strategy is professionally suitable even if it produces tax savings.

3.3. Deterministic Simulation Engine

The deterministic simulation engine is implemented by `compute_tax()`. It receives the taxpayer profile and an optional set of strategy adjustments. It computes income, AGI, deductions, QBI deduction, taxable income, ordinary tax, capital-gains tax, NIIT, self-employment tax, credits, AMT-style tax, effective rate, marginal rate, and balance due. The function returns a structured `TaxResult` dataclass.

This is the computational center of the notebook. The LLM does not compute tax savings directly. Instead, each proposed strategy is transformed into an adjustment dictionary, and `compute_tax()` determines its numerical effect. This separation between language generation and tax calculation is essential for governance. It allows a reviewer to inspect the exact fields that changed and the exact arithmetic that produced the result.

3.4. TaxOptimizationLoop

`TaxOptimizationLoop` executes one research cycle. It is responsible for generating strategies, running legal gates, simulating all candidate strategies, selecting the best admissible candidate, stress-testing that candidate, obtaining feedback, and generating search seeds for the next cycle. The result is stored in a `TaxCycleResult` dataclass.

The class is designed around structured state. The output of each cycle contains the generated strategies, simulation results with legality flags, the best strategy, the best deterministic result, baseline tax, estimated savings, the stress-test table, fragility flag, feedback verdict, and next-cycle seeds. The dataclass also contains audit-risk and legal-verdict fields, although the current code primarily carries those values inside the strategy and simulation records. This record functions as an audit artifact. It preserves both positive and negative evidence and prevents the process from relying on informal memory.

3.5. HumanAdvisorAgent

`HumanAdvisorAgent` simulates Margaret Chen, a senior CPA with 25 years of experience. It receives the best strategy and cycle result, then returns a structured professional judgment: `APPROVE`, `APPROVE_WITH_MODS`, or `REJECT`. It also provides a first-person rationale, required modifications, three client talking points, and remaining concerns.

This layer represents professional skepticism. A tax strategy may pass a synthetic legal gate and produce simulated savings but still require advisor review. It may be too complex, too aggressive, too difficult to explain, or too fragile under audit. The human-advisor layer therefore models the professional responsibility dimension of the workflow.

3.6. TaxEvaluationAgent

`TaxEvaluationAgent` is the outer governor. It controls the maximum number of planning rounds, stores decision logs, accumulates failure context, and determines whether the process should continue. It combines deterministic heuristics with an LLM-supported decision protocol. The key decisions are: `CONTINUE`, `STOP_SUCCESS`, and `STOP_MAX_ROUNDS`.

The outer agent prevents uncontrolled optimization. Without such a governor, an agentic tax-planning system could continue generating strategies until it finds one that appears favorable by chance or by exploiting a modeling artifact. The implementation sets `MAX_ROUNDS = 3`, `MIN_SAVINGS = 3000`, and `MAX_AUDIT_RISK = low`. The heuristic stops only when the CPA approves or approves with modifications, savings meet the threshold, the strategy is not fragile, and the generated audit-risk estimate is `low` or `medium`; otherwise it continues or stops at the maximum-round boundary.

4. Mathematical and Computational Core

The mathematical core defines the quantities that the system is allowed to compute and compare. This is important because LLMs are fluent in tax language but must not be permitted to invent unimplemented calculations. The formal layer specifies income aggregation, deductions, taxable income, ordinary tax, capital-gains tax, surtaxes, credits, and the strategy-comparison objective.

4.1. Income Aggregation

Let ordinary wage income be denoted by W, consulting or self-employment income by S, rental net income by R, qualified dividends by D_q, short-term capital gains by G_s, and long-term capital gains by G_l. The gross income measure used by the simulator can be represented as

$$GI = W + S + R + D_q + G_s + G_l. \tag{1}$$

The model then applies above-the-line adjustments such as deductible retirement contributions, HSA contributions, deductible IRA contributions, and other permitted adjustments. Let A denote total allowable above-the-line adjustments. Adjusted gross income is

$$AGI = GI - A. \tag{2}$$

This formulation is intentionally modular. A candidate strategy does not rewrite the tax engine. It changes one or more components of the taxpayer profile, and the deterministic engine recomputes the outcome.

4.2. Deductions

Let D_s denote the standard deduction and D_i denote allowable itemized deductions. The simulator chooses the larger amount:

$$D = \max(D_s, D_i). \tag{3}$$

Itemized deductions include allowable mortgage interest, charitable deductions, and capped state and local tax deductions. If $SALT$ denotes state and local taxes and $\overline{SALT}$ denotes the statutory cap in the synthetic code, the allowable SALT deduction is

$$D_{SALT} = \min(SALT, \overline{SALT}). \tag{4}$$

The charitable deduction may also be capped as a fraction of AGI. If C is cash charitable giving and λ_C is the AGI-based limit, then

$$D_C = \min(C, \lambda_C AGI). \tag{5}$$

These caps make the planning environment nonlinear. Increasing a deductible expense does not always reduce taxable income dollar-for-dollar. This is why deterministic simulation is necessary.

4.3. Taxable Income and QBI Deduction

Let Q denote the allowable qualified business income deduction. The simulator computes taxable income as

$$TI = \max(0, AGI - D - Q). \tag{6}$$

The QBI deduction depends on business income, taxable income thresholds, and synthetic limitations encoded in the tax code. The important governance point is that the deduction is calculated by the engine, not by the LLM. Candidate strategies may affect business income or related deductions, but the rule application remains deterministic.

4.4. Progressive Ordinary Tax

Ordinary tax is computed using a progressive bracket schedule. For brackets indexed by b, with lower bound L_b, upper bound U_b, and marginal rate τ_b, ordinary tax can be written as

$$T_o(TI_o) = \sum_b \tau_b \cdot \max\left(0, \min(TI_o, U_b) - L_b\right), \tag{7}$$

where TI_o is the portion of taxable income treated as ordinary income. This structure captures the marginal nature of the tax schedule. A planning strategy may reduce total taxable income, but the value of the reduction depends on which bracket the taxpayer occupies.

4.5. Capital-Gains Tax

Long-term capital gains and qualified dividends are taxed under a separate rate schedule. Let $K = G_l + D_q$ denote preferential income. The capital-gains tax is computed using the synthetic long-term capital-gains brackets:

$$T_k(K) = \sum_b \kappa_b \cdot \max\left(0, \min(K, V_b) - M_b\right), \tag{8}$$

where κ_b is the preferential rate and $[M_b, V_b]$ is the relevant capital-gains bracket. The separation between ordinary income and preferential income allows the simulation to test strategies involving long-term gains, harvested losses, and income timing.

4.6. Total Tax, Credits, and Balance Due

Let T_{SE} denote self-employment tax, T_{NIIT} denote net investment income tax, T_{AMT} denote the incremental AMT-style add-on computed by the notebook, and CR denote credits. The implementation first calculates tentative AMT and then stores only the positive excess over ordinary and capital-gains tax. Total tax before payments is therefore

$$T = \max\left(0, T_o + T_k + T_{NIIT} + T_{SE} + T_{AMT} - CR\right). \tag{9}$$

If withholding is WH, the balance due reported by the notebook is

$$B = T - WH. \tag{10}$$

The model also computes the effective tax rate,

$$ETR = \frac{T}{GI}, \tag{11}$$

when gross income is positive. These statistics allow the dashboard to compare baseline and optimized outcomes.

4.7. Strategy Evaluation Objective

For a candidate strategy h, let T_0 denote baseline tax and T_h denote tax after applying the strategy adjustments. Estimated savings are

$$S_h = T_0 - T_h. \tag{12}$$

A cycle's provisional best strategy is selected by sorting simulated strategies by S_h after excluding only those marked `ILLEGAL`; `BORDERLINE` strategies can still be simulated. The code prefers legal strategies with positive savings, and falls back to the best legal strategy if none saves tax. The later decision to stop is governance-constrained:

$$h^\star = \arg\max_{h \in \mathcal{H}_{\text{not illegal}}} S_h \quad \text{followed by risk, stress, and advisor-review controls.} \tag{13}$$

This equation captures the central idea of the notebook. Tax planning is not unconstrained optimization. It is constrained professional discovery.

5. The TaxEvaluationAgent Decision Protocol

The `TaxEvaluationAgent` formalizes the continuation problem. In many advisory settings, the decision to continue searching, accept a recommendation, or stop is made informally. This creates several risks: the advisor may overweight a large savings number, ignore fragility, repeat failed ideas, or fail to document why a strategy was accepted. The evaluation agent makes this decision explicit.

5.1. Motivation

The motivation for a separate evaluation agent is governance discipline. The component that generates strategies should not be the same component that decides whether the search process is complete. The strategy generator is designed to explore. The evaluation agent is designed to control. This separation mirrors the institutional distinction between research, review, and approval.

The agent applies three broad criteria. First, the strategy must be materially useful, meaning that simulated savings must exceed a minimum threshold. Second, the strategy must be acceptable under risk and fragility constraints. Third, the strategy must survive professional-style review. A strategy that fails any of these conditions should generally trigger another round or be rejected.

5.2. Two-Step Protocol

The decision protocol has two stages. The first is a deterministic heuristic. If the CPA-style reviewer rejects the strategy, the system continues. If the strategy is fragile under stress testing, the system continues. If savings are below $3,000, the system continues. If savings are material, the CPA approves or approves with modifications, the generated audit-risk estimate is `low` or `medium`, and the strategy is not fragile, the heuristic recommends stopping successfully. Although the class stores `MAX_AUDIT_RISK = low`, the current heuristic admits both low and medium strategy-level risk estimates.

The second stage is a constrained LLM decision. The evaluation prompt receives the round number, best strategy, savings, audit risk, stress fragility, AI feedback, CPA verdict, CPA rationale, proposed modifications, and heuristic recommendation. It must return a structured decision object:

```
{
"decision": "CONTINUE | STOP_SUCCESS | STOP_MAX_ROUNDS",
"reasoning": "short explanation grounded in savings, risk, stress, and review",
"failure_summary": "context to inject into the next cycle"
}
```

The schema is intentionally narrow. The model may reason about the evidence, but it may not modify the software contract. This preserves procedural integrity while allowing semantic interpretation.

5.3. Failure Context

A key feature of the protocol is failure-context accumulation. The current code adds the best strategy from each continuing round to `rejected_names`, records the CPA verdict, required modifications, LLM failure summary, and next-cycle seed names, and passes that text into the next `TaxOptimizationLoop`. This reduces the risk that the LLM regenerates the same strategy under a different name. It also turns negative evidence into useful search guidance.

In an AI4S framework, failure is not an embarrassment to hide; it is an experimental result. A strategy that fails because it is audit-fragile, liquidity-demanding, legally unsupported, or too complex teaches the system where not to search next. The `TaxEvaluationAgent` is the mechanism that converts failed planning attempts into structured memory.

6. Legal, Numerical, and Governance Safeguards

Automated tax-planning systems require safeguards because they operate in a high-stakes professional domain. An erroneous recommendation can create financial cost, legal exposure, reputational damage, and professional liability. The notebook therefore implements safeguards at several levels: legal environment, strategy representation, computation, stress testing, professional review, and stopping logic.

6.1. Legal Environment Safeguards

The synthetic tax code defines a bounded legal environment. This prevents the LLM from relying on unsupported legal claims. The legal validity gate then reviews each candidate strategy against that environment. Strategies can be rejected before simulation if they are outside the allowed rules, inconsistent with taxpayer facts, or too vague to evaluate.

This is not equivalent to real legal review. Rather, it is a model of how legal review should be embedded in an AI-assisted workflow. In a production system, the synthetic code would be replaced or supplemented by validated tax law, jurisdiction-specific rules, firm policy, and licensed professional review.

6.2. Structured Strategy Safeguards

The strategy-generation prompt requires structured JSON output. Each strategy must contain a name, two-sentence description, adjustment dictionary, rationale of at most 20 words, estimated audit-risk label, and liquidity-impact estimate. The adjustment dictionary is especially important. It forces the strategy to be expressed as changes to known taxpayer fields, rather than as an unbounded paragraph.

Structured output reduces ambiguity. A strategy that says "optimize retirement planning" is not directly testable. A strategy that increases retirement contributions by a specified amount is testable. The model therefore converts language into a computational object before evaluation.

6.3. Numerical Safeguards

The deterministic tax engine protects against hallucinated numbers. It computes results from encoded rules and taxpayer data. It also decomposes the result into components: gross income, AGI, taxable income, ordinary tax, capital-gains tax, NIIT, self-employment tax, AMT, credits, total tax, effective rate, marginal rate, QBI deduction, itemized deductions, and balance due. This decomposition makes the output inspectable.

A production version would require additional controls, including unit tests, regression tests, cross-checks against authoritative calculators, edge-case testing, versioned tax-law tables, and independent model validation. The notebook demonstrates the architecture rather than a production tax engine.

6.4. Professional-Review Safeguards

The `HumanAdvisorAgent` models professional skepticism. Its simulated CPA review examines not only savings but also suitability, audit risk, complexity, modifications, client talking points, and remaining concerns. This layer is essential because many tax strategies are not binary. They may be acceptable only if modified, documented, phased, or explained carefully to the client.

The review layer also makes the client-facing dimension explicit. A strategy that cannot be clearly explained is not ready for deployment. A strategy that requires the client to take actions inconsistent with stated preferences is professionally suspect. The human-advisor layer captures these considerations.

6.5. Stopping Safeguards

The maximum number of rounds prevents uncontrolled search. In the uploaded code, the limit is three rounds. The minimum savings threshold, $3,000, prevents the system from celebrating immaterial improvements. The audit-risk check prevents the system from maximizing savings at the expense of defensibility, though the current heuristic treats both low and medium generated audit-risk labels as acceptable. The stress-fragility rule prevents the system from advancing strategies that work only in the base case. Together, these controls convert AI-assisted search into governed professional exploration.

7. LLM Integration

The LLM is integrated as a constrained reasoning and generation component. It is not treated as a tax authority, calculator, or final decision-maker. Its primary functions are strategy generation, legal validity review, stress-test interpretation, feedback, next-cycle seed generation, CPA-style review, and evaluation-agent reasoning. Each function is governed by a prompt contract and expects structured output.

The notebook uses Anthropic Claude through a centralized function. In the uploaded code, the model identifier is stored in `MODEL`:

```
MODEL = 'claude-haiku-4-5-20251001'
```

The wrapper function is:

```
ask_claude(system, user, max_tokens=1200)
```

This function sends system and user prompts to the model and returns text. The Colab setup installs `anthropic`, retrieves `ANTHROPIC_API_KEY` from `google.colab.userdata`, and constructs an `anthropic.Anthropic` client. A separate parsing utility removes common Markdown code fences and parses JSON. This design is important because LLM outputs can be linguistically rich but structurally unstable. The surrounding code must therefore enforce the interface.

The implementation uses six Claude prompt families:

1. **Strategy generation.** Claude proposes exactly five tax-planning strategies expressed as structured adjustment dictionaries.
2. **Legal validity gate.** Claude reviews whether each strategy is `LEGAL`, `BORDERLINE`, or `ILLEGAL` inside the synthetic tax-code environment.
3. **Best-strategy feedback.** Claude evaluates the selected strategy after deterministic stress testing and returns a short robustness diagnosis.
4. **Next-cycle seed generation.** Claude proposes two refined search directions based on feedback and rejected strategy names.
5. **CPA-style review.** Claude simulates Margaret Chen, CPA, and returns an approval, approval-with-modifications, or rejection object.
6. **Evaluation-agent decision.** Claude helps the outer agent decide whether to continue, stop successfully, or stop at the maximum-round boundary.

The most important design principle is that LLM judgment is never allowed to replace deterministic computation. The tax result comes from `compute_tax()`, not from narrative inference. The LLM can help search the strategy space, but the engine computes the effect of each strategy. This separation is the core technical safeguard of the architecture.

The model can be replaced without changing the architecture. A stronger or more specialized LLM may generate better strategies or more nuanced reviews, but it would still operate under the same governance contract: structured outputs, deterministic calculation, legal gating, stress testing, professional review, and outer-agent decision control.

8. Stress Testing Scenarios

Stress testing asks whether a candidate tax strategy remains credible when important assumptions change. A strategy that is attractive under the baseline taxpayer profile may become weak if income rises, deductions are capped, or tax rates change. The uploaded notebook therefore subjects the best strategy in each cycle to three deterministic stress scenarios.

The scenarios are illustrative rather than exhaustive. Their purpose is to model the governance principle that a recommendation should not be judged only in the base case. A production system would include jurisdiction-specific policy changes, multi-year projections, state tax interactions, entity-level effects, liquidity modeling, audit-risk tightening, and documentation requirements.

Scenario A — Rate Increase. The top ordinary brackets above 30 percent are increased by four percentage points. The same strategy adjustments are recomputed against this modified `TAX_CODE`. This tests whether the strategy remains useful when higher rates apply.

Scenario B — Deduction Cap. The 401(k) contribution limit and the SEP-IRA absolute limit are reduced by 20 percent. The same strategy adjustments are recomputed under those lower limits. This tests whether the strategy depends too heavily on retirement-contribution deductions.

Scenario C — Income Spike. Freelance income is increased by $30,000 while the same strategy adjustments remain in force. Savings are measured against a new baseline for the higher income case. This tests whether the recommendation remains effective when consulting income rises.

In the code, a strategy is marked fragile if the minimum savings across these three stress scenarios falls below $500. Fragility does not necessarily mean that the strategy has no value. It means that the strategy should not be accepted without modification, documentation, or additional review.

9. Discussion and Limitations

The architecture should be evaluated as a governance design, not as a claim of real-world tax-law completeness. Its primary strength is the explicit connection between generative AI, deterministic simulation, legal screening, stress testing, professional review, and decision logging. Its primary limitation is that the tax environment is synthetic and simplified.

9.1. Contributions

The first contribution is a formal mapping from AI4S to tax planning. The paper shows how observation, representation, hypothesis generation, simulation, validation, feedback, and revision can be implemented as a professional advisory workflow. The second contribution is the separation between `TaxOptimizationLoop` and `TaxEvaluationAgent`. This separation prevents the strategy generator from controlling the final governance decision. The third contribution is the explicit human-in-the-loop layer, represented by the `HumanAdvisorAgent`. The fourth contribution is the insistence that tax outcomes be computed deterministically rather than accepted as LLM claims.

9.2. Limitations

The most important limitation is that the tax code is synthetic. Real tax planning would require actual federal, state, local, and possibly international rules. It would also require jurisdiction-specific authority, current-year updates, entity classification, filing-status nuances, carryforward rules, depreciation schedules, basis tracking, passive activity rules, estimated tax safe harbors, penalty calculations, and professional documentation.

A second limitation is that the human advisor is simulated by an LLM. This is useful for prototyping, but it is not a substitute for licensed professional review. In a real advisory practice, the system should escalate candidate strategies to a human CPA, tax attorney, or enrolled agent before client implementation.

A third limitation is stochasticity. LLM-generated strategies may vary across runs. This creates both opportunity and risk. Diversity can improve search, but it also requires logging, versioning, reproducibility controls, and seed management. A production system would need to store prompts, responses, model versions, tax-code versions, taxpayer-data versions, and computation hashes.

A fourth limitation is that the model currently evaluates single-year planning effects. Many tax strategies are inherently multi-year. Retirement contributions, charitable bunching, capital-loss harvesting, Roth conversions, entity structuring, depreciation decisions, and income deferral may create future tax effects that are not captured by a one-year simulation.

9.3. Future Work

Future work should proceed along five lines. First, the synthetic tax code should be replaced with validated, versioned tax-law modules. Second, the simulation engine should be expanded to multi-year projections. Third, the legal gate should be strengthened with citations to authoritative sources, firm policy, and jurisdiction-specific rules. Fourth, the human-advisor layer should be connected to actual professional review workflows. Fifth, the dashboard should export model cards, risk registers, approval checklists, client memos, reproducibility manifests, and audit bundles.

A particularly important extension is audit documentation. In tax and accounting, a recommendation is not only a numerical output. It must be supported by facts, law, reasoning, calculations, documentation, and review. The next stage of this architecture should therefore move from a notebook prototype to an institutional audit-control system.

10. Conclusion

This paper has presented an AI4S architecture for agent-governed tax planning. The system combines a synthetic tax code, a detailed taxpayer profile, deterministic tax simulation, LLM-generated strategy search, legal validity gates, stress testing, CPA-style review, and a supervisory evaluation agent. Its central design principle is that AI creativity must be embedded inside professional governance.

The architecture is not a tax chatbot. It is a controlled discovery loop. The LLM proposes strategies, but does not compute the tax result. The deterministic engine computes the result, but does not decide professional suitability. The legal gate screens validity, but does not replace human review. The simulated CPA introduces professional skepticism, but does not erase the need for real advisor judgment. The evaluation agent coordinates these layers and records why the process continues or stops.

The broader implication is that AI can contribute meaningfully to tax, accounting, and advisory services when it is implemented as a governed system rather than an unconstrained answer generator. The future of AI in professional services should not be understood as automation without oversight. It should be understood as disciplined augmentation: broader search, faster simulation, clearer documentation, stronger stress testing, and more explicit review.

The companion notebook makes this claim operational. It shows how one tax-planning case can be converted into a structured AI4S loop: generate, simulate, govern, review, stress-test, decide, and iterate. The result is a method for making advisory reasoning more transparent and accountable. Its value lies not in replacing judgment, but in making judgment more structured, auditable, and professionally defensible.

Appendix A. Companion Computational Notebook

The paper is accompanied by one computational notebook:

Open the companion Colab notebook

The notebook implements the architecture described in this paper. It contains a complete, self-contained demonstration of a governed agentic tax-planning workflow. The executable implementation is organized into nine cells, preceded by introductory Markdown:

1. Setup, dependencies, Claude client, and parsing utilities.
2. Synthetic tax code.
3. Taxpayer profile.
4. Deterministic tax simulation engine.
5. `TaxOptimizationLoop`.
6. `HumanAdvisorAgent`.
7. `TaxEvaluationAgent`.
8. Orchestrator and execution entry point.
9. Final report dashboard and visual audit trail.

The notebook produces a final dashboard and visual audit trail. The workflow diagram in Appendix Appendix B summarizes the same governed process without relying on implementation filenames in the manuscript text.

The notebook should be interpreted as a methodological prototype. It does not represent actual legal advice, and its synthetic tax code is not a substitute for current tax law. Its purpose is to show how AI-assisted professional reasoning can be governed through explicit rules, deterministic simulation, structured agent outputs, legal gates, stress tests, advisor review, and decision logs.

Appendix B. Workflow Diagram

Figure B.1 provides the workflow diagram corresponding to the agent-governed AI4S tax-planning architecture described in Section 2. It summarizes the movement from taxpayer facts and tax-code representation to strategy generation, legal screening, deterministic simulation, stress testing, professional review, and evaluation-agent decision.

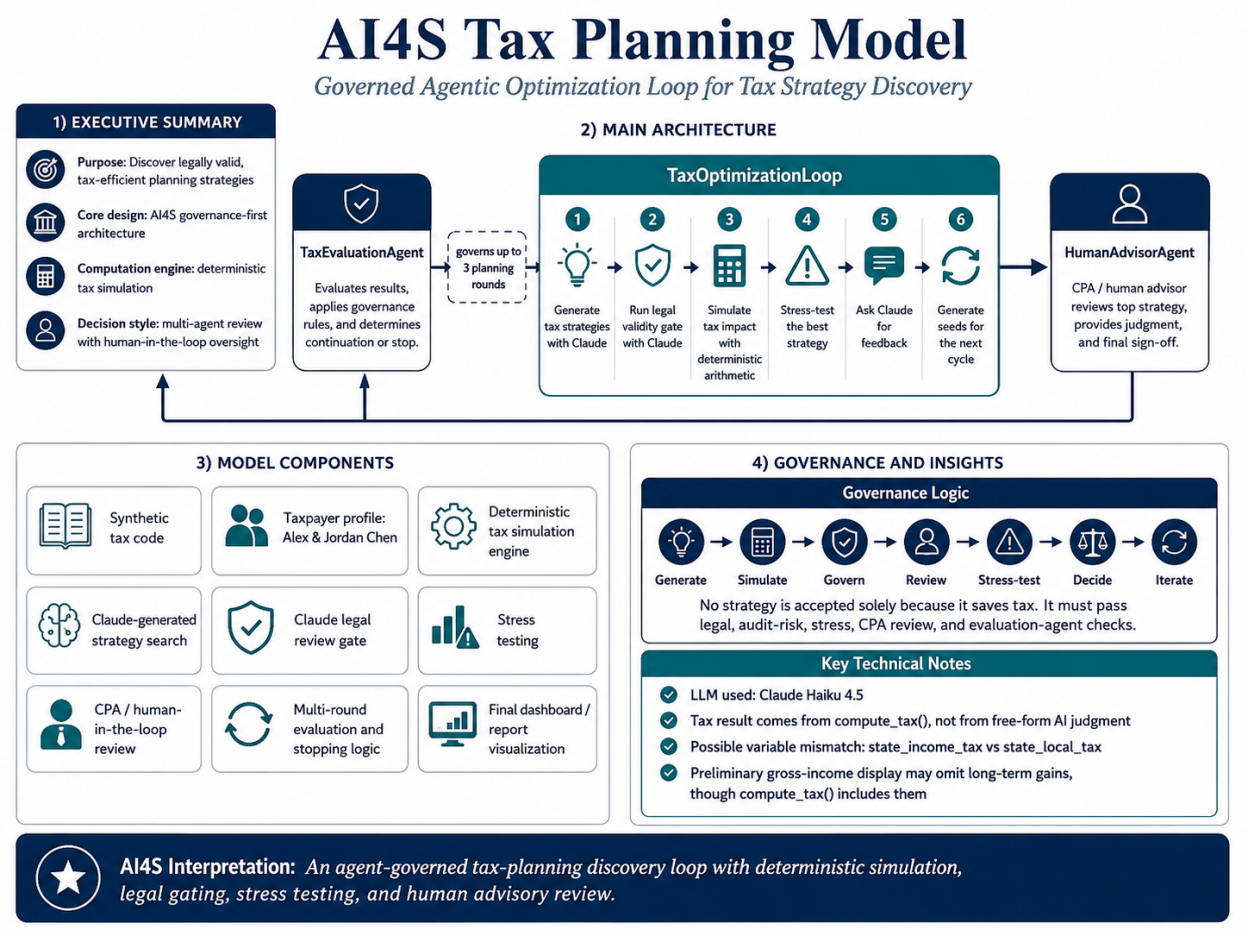

Figure B.1: Workflow diagram for the agent-governed AI4S tax-planning and audit-review architecture.

References

[1] H. Wang *et al.*, "Scientific discovery in the age of artificial intelligence," *Nature*, vol. 620, pp. 47–60, 2023.

[2] J. Jumper *et al.*, "Highly accurate protein structure prediction with AlphaFold," *Nature*, vol. 596, pp. 583–589, 2021.

[3] K. T. Butler, D. W. Davies, H. Cartwright, O. Isayev, and A. Walsh, "Machine learning for molecular and materials science," *Nature*, vol. 559, pp. 547–555, 2018.

[4] B. Burger *et al.*, "A mobile robotic chemist," *Nature*, vol. 583, pp. 237–241, 2020.

[5] S. Russell, *Human Compatible: Artificial Intelligence and the Problem of Control*. Penguin, 2020.

[6] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework*. NIST AI 100-1, 2023.

[7] American Institute of Certified Public Accountants, *Trust Services Criteria for Security, Availability, Processing Integrity, Confidentiality, and Privacy*. AICPA, 2017.

[8] Internal Revenue Service, *Publication 17: Your Federal Income Tax*. U.S. Department of the Treasury, 2024.

[9] Internal Revenue Service, *Publication 505: Tax Withholding and Estimated Tax*. U.S. Department of the Treasury, 2024.